动态LoRA高效微调

完整项目标题：基于动态参数分配的 LoRA 大语言模型高效微调优化方法

赛题一：算法模型创新

一、项目概述

本项目面向大语言模型低资源微调场景，针对传统 LoRA 对所有层使用相同 rank、造成参数分配不均的问题，提出一种动态参数分配方法。系统在训练过程中估计各层重要性，在固定总 rank
预算下，将更多可训练参数分配给重要层，从而提高参数利用率并降低显存和计算开销。

二、问题定义与创新点

LoRA 将权重更新写为 ΔW=B A。传统方案固定每个目标层的
rank，忽略不同层对任务的敏感程度。本项目的创新点包括：1）基于 LoRA
梯度和更新幅度估计层重要性；2）用指数滑动平均降低单步噪声；3）在总 rank
预算不变时动态调整各层 rank；4）采用最大 rank 参数矩阵切片，调整 rank 时无需重新初始化。

三、算法设计

训练反向传播后，对第 l
层计算重要性：I_l=EMA(|∂L/∂A_l| · mean(|B_l|)+|∂L/∂B_l| · mean(|A_l|))。每隔固定步数按照 I_l
从高到低分配 rank，并满足
sum(r_l)=R、r_min≤r_l≤r_max。重要层得到更大的低秩子空间，不重要层减少参数。核心实现见
dynamic_lora.py。

四、实验设计与结果

实验对比均匀 LoRA 与动态 LoRA，使用相同随机种子、网络、优化器、训练步数和总 rank
预算。当前本地小测试仅用于验证 CUDA、动态 rank 和日志流程，不作为正式成绩数据。正式实验将在云端使用真实大语言模型和公开数据集，记录验证损失、任务准确率、显存、训练时间、可训练参数量，并与固定 rank LoRA 对比。

五、应用价值

该方法适合显存受限的个人电脑、边缘设备和个性化模型微调，可作为 Qwen、LLaMA、BERT 等模型的 LoRA 优化插件。通过固定预算动态调度，能够在效果和资源消耗之间取得更好的平衡。

六、总结与展望

项目完成了动态重要性评估、rank 预算约束、参数切片和对比实验闭环。后续将接入公开指令数据集，增加准确率、显存、吞吐量和多基线实验，并研究更平滑的 rank 变化策略。

七、复现方式

安装 requirements.txt 后运行 python experiment.py；最简演示运行 python simple_train.py --steps 100。代码、配置和实验结果已随本方案放在同一提交目录。

附录：提交材料说明

代码文件：dynamic_lora.py、simple_train.py、experiment.py；运行说明：README.md。云端实验完成后再补充正式结果和图表。提交前需录制 3—5 分钟演示视频，并按参赛团队编号统一命名。